

Notes for ECE 50877 - Introduction to Applied Cryptography

Zeke Ulrich

September 1, 2026

Contents

<i>Course Description</i>	1
<i>Basis of Cryptography</i>	2
<i>Computational Security</i>	4
<i>Pseudorandom Number Generators</i>	5

Course Description

Cryptography is an essential component of securing communication and data in a wide range of applications. This course explores fundamental building blocks in cryptography such as ciphers, hash functions, and digital signatures. We will then demonstrate how these building blocks are used practically to secure communications and systems. We will also learn about the power of advanced cryptography to enable complex functionalities with rigorous security guarantees. We will cover fundamental secure multiparty computation protocols, homomorphic encryption, and blockchains, and review their application in building privacy-preserving machine learning. We will also explore social and ethical issues in developing and deploying cryptographic systems.

Basis of Cryptography

Consider the archetypical case in cryptography. Alice wants to send a message to Bob, and wants the contents to remain secret from Vincent. Alice encrypts her message with a secret key s .

An encryption scheme (Gen, Enc, Dec) is a triple of algorithms where

- Key generation $\text{Gen}(1^n)$ outputs a secret key k of length n (also called the security parameter).
- Encryption algorithm $\text{Enc}_k(m) = c$ outputs ciphertext c .
- Decryption algorithm $\text{Dec}_k(c) = m$ outputs plaintext m .

An encryption algorithm is defined over a message space M and a key space K . Requirements:

- Correctness: $\text{Dec}_k(\text{Enc}_k(m)) = m$.
- Security: definition with respect to adversaries.

The secret key k should be hidden from the adversary. The encryption and decryption algorithms themselves should be public; this is Kerckhoffs's principle. This is because the secrecy keys are easy to hide and replace in case of a breach, not so with algorithms. It is also good to have many eyes on an algorithm to expose security flaws.

The key space must be large enough such that a brute force attack is infeasible. Consider a Caesar cipher; the key space is only 26 long and a message so encrypted is quickly bruteforced.

Shannon's Treatment: messages come from some distribution, let M be a random variable for sampling the messages from the message space M . Distribution M is known to the adversary. This captures a priori information about the messages. The ciphertext $c = \text{Enc}_k(m)$ depends on m chosen according to M . k chosen randomly according to Gen. Enc may also use some randomness. Dec is deterministic. These induce a distribution C over the ciphertexts c . The adversary only observes c . Knowledge about m before observing the output of C is captured by M . Knowledge about m after observing the output of C is captured by $M|C$. An encryption scheme (Gen, Enc, Dec) is *Shannon secure* if for every probability distribution for M , every message $m \in M$, and every ciphertext $c \in C$: $\Pr[M = m|C = c] = \Pr[M = m]$.

An alternative definition of secrecy is *Perfect Secrecy*. An encryption scheme is perfectly secure for every pair of messages m_1 and $m_2 \in M$ and every ciphertext $c \in C$ if $\Pr[\text{Enc}_k(m_1) = c] = \Pr[\text{Enc}_k(m_2) = c]$.

Shannon had fingers in many cryptographic pies. Shannon's Theorem says that if (Gen, Enc, Dec) is a perfectly secret encryption scheme with message space M and key space K , then $|K| \geq |M|$. This means

that for an encryption scheme to be perfectly secret, the keys must be as long as the message. The challenge of modern cryptography is to make something just as good as perfect for practical purposes.

Computational Security

A scheme is (t, ε) -secure if any adversary running for time at most t succeeds in breaking the scheme with probability at most ε . However, this definition makes no guarantees for after t ; perhaps the probability of an attacker succeeding rises sharply after t .

A private-key encryption scheme with key length n and for an adversary running for time t , succeeds in breaking the scheme with probability at most $c \frac{t}{2^n}$ (for some constant c).

A better approach to security is to say a scheme is secure if any *efficient* adversary succeeds in breaking the scheme with at most *negligible* probability.

A function f (from natural numbers to non-negative real numbers) is polynomially bounded (or polynomial) if there is a constant c such that $f(n) < n^c$ for all n .

An algorithm A runs in polynomial time if there is a polynomial p s.t. for all inputs $x \in \{0, 1\}^*$, $A(x)$ terminates within $p(|x|)$ steps.

An algorithm is efficient if it runs in polynomial time.

A function $v(n)$ is negligible if for every c , there exists some N such that for all $n > N$, $v(n) \leq \frac{1}{n^c}$.

A scheme is computationally secure if for every probabilistic polynomial-time (PPT) adversary A carrying out an attack, the probability that A succeeds is negligible.

Pseudorandom Number Generators

Cryptography needs a lot of random to be secure. Computers supply a little bit through core temperature, power consumption, mouse movement, and it would be nice to expand those random bits into many more random bits.

Specifically, given n uniformly random bits $x = x_1|x_2|\dots|x_n$, we want a deterministic polynomial-time algorithm G such that $G(x)$ outputs $l(n)$ bits $y = y_1|y_2|\dots|y_{l(n)}$ with $l(n) > n$ and y looks "as good as" a truly random string. $G : \{0, 1\}^n \rightarrow \{0, 1\}^{l(n)}$ is called a pseudorandom generator (PRG). "As good as" means that the string will pass all statistical tests that a truly random string would pass. You can never generate more information from nothing, so the idea of "as good as" means that no efficient computer can tell $G(x)$ and a truly random string apart. The distributions

$$\left\{x \leftarrow \{0, 1\}^{l(n)} : G(x)\right\} \quad (1)$$

and

$$\left\{r \leftarrow \{0, 1\}^{l(n)} : r\right\} \quad (2)$$

are "computationally indistinguishable".

"Computationally indistinguishable" means that no PPT distinguisher algorithm D can tell them apart.

A deterministic algorithm G is called a pseudorandom generator if:

- G can be computed in polynomial time.
- $|G(x)| > |x|$
- $\{x \leftarrow \{0, 1\}^n : G(x)\} \approx_c \left\{U_{l(n)}\right\}$ where $l(n) = |G(0^n)|$

$\approx_c$ denotes computational indistinguishability. $U_{l(n)}$ denotes uniform distribution over $\{0, 1\}^{l(n)}$. $|G(x) = l(n)|$ is the expansion factor of G .